

Session Checkpoint

Sink Consumers, Schema Redesign & Pipeline Debugging

Real-Time Anomaly Detection Platform

July 13–14, 2026

Contents

1	Session Objectives	2
2	Architectural Discoveries	2
2.1	Dashboard Cannot Consume from Kafka Directly	2
2.2	FastAPI as First-Layer Enforcement	2
2.3	Streaming Pipeline as State Maintenance Loop	2
3	Schema Redesign	3
3.1	Transactions Table	3
3.2	Evaluation Labels Table (New)	3
3.3	Alerts Table (Rebuilt)	3
3.4	Load Script Updated	3
4	Sink Consumers	3
4.1	Architecture	3
4.2	Raw-Events Sink	4
4.3	Decisions Sink	4
4.4	Profile Update Rules	4
4.5	DecisionCore Output Change	4
5	Infrastructure Changes	4
5.1	Docker Compose	4
5.2	Dockerfile	4
5.3	Prometheus	5
5.4	Batch Scripts	5
6	Debugging Journey & Findings	5
6.1	Stale PostgreSQL Connection	5
6.2	Local vs Docker PostgreSQL Conflict	5
6.3	Missing Profiles After Volume Wipe	5
6.4	Buffer Contamination from Multiple Simulator Runs	5
6.5	Feature Distribution Mismatch (Open)	6
7	Updated Roadmap	6
8	Files Modified / Created	6

1 Session Objectives

The session began with the goal of building the Streamlit supervisor dashboard. During the design phase, several fundamental architectural gaps were uncovered that required resolution before the dashboard could function correctly. The session pivoted to addressing these gaps.

1. Design the data path from the streaming pipeline to the dashboard
2. Build sink consumers to persist pipeline output to PostgreSQL and Redis
3. Redesign the database schema for streaming compatibility
4. Validate the full pipeline end-to-end with the new components

2 Architectural Discoveries

2.1 Dashboard Cannot Consume from Kafka Directly

Streamlit's execution model re-runs the entire script on every user interaction (button click, filter change). A Kafka consumer is a stateful, long-running process that maintains offsets and group membership. Embedding a consumer inside a Streamlit script would:

- Lose offset position between re-runs
- Miss messages arriving between user interactions
- Create consumer group conflicts with multiple simultaneous users

Decision: Decoupled sink consumers write to PostgreSQL; the dashboard reads from PostgreSQL via periodic polling. Given the alert volume (78/day across 150 branches), polling latency is acceptable for the supervisory overview use case.

2.2 FastAPI as First-Layer Enforcement

A critical architectural insight emerged: the FastAPI simulation API serves as the **synchronous enforcement layer**. In production, the teller's workstation calls the API before finalizing an operation, receiving an instant ALLOW/BLOCK/REVIEW verdict. The dashboard's role is supervisory oversight, not real-time enforcement.

2.3 Streaming Pipeline as State Maintenance Loop

The streaming pipeline's primary value is **continuous state maintenance**, not duplicate enforcement. Without it:

- Client profiles in Redis go stale between nightly batch runs
- The LSTM sequence buffer never receives new events
- Short-term counters (24h tx count, cumulative amount) freeze
- Sequential patterns like smurfing across multiple transactions are missed

The FastAPI endpoint reads state; the streaming pipeline writes state.

3 Schema Redesign

3.1 Transactions Table

Removed oracle label columns (`is_anomaly`, `anomaly_type`) from the `transactions` table. These are training/evaluation artifacts that have no meaning in a streaming context where ground truth is unknown at event time. Foreign keys to master tables retained — in production, the core banking system guarantees master records exist before transactions flow.

3.2 Evaluation Labels Table (New)

```
CREATE TABLE evaluation_labels (
  event_id UUID PRIMARY KEY REFERENCES transactions(event_id),
  is_anomaly BOOLEAN NOT NULL,
  anomaly_type VARCHAR(30)
);
```

Batch-only table. FK to `transactions` is safe because batch loads transactions first, then labels. Clean separation: `transactions` holds facts, `evaluation_labels` holds ML ground truth.

3.3 Alerts Table (Rebuilt)

IMPORTANT: No FK to `transactions(event_id)`. In the streaming path, the raw-events sink and decisions sink consume from different topics in parallel. A FK would create a race condition: the decisions sink may arrive before the raw-events sink inserts the transaction row. Referential integrity is guaranteed by pipeline topology — a decision event cannot exist without a corresponding raw event having entered the pipeline. This is a deliberate architectural decision, not an oversight.

New columns added: `branch_id`, `amount`, `operation_type`, `regulatory_flags` (JSONB), `reasons` (JSONB), `created_at`. Indexes on `branch_id`, `tier`, `timestamp`, `operation_type` for dashboard query patterns.

3.4 Load Script Updated

`scripts/load_postgresql.py` updated to write `transactions` without oracle columns and populate `evaluation_labels` separately from the same CSV source.

4 Sink Consumers

4.1 Architecture

Two new long-running consumers added to the streaming pipeline:

Consumer	Topic	Writes To	Port
Raw-Events Sink	<code>raw-events</code>	PostgreSQL <code>transactions</code>	9103
Decisions Sink	<code>decisions</code>	PostgreSQL <code>alerts</code> + Redis	9104

Both use independent consumer groups (separate from the transform-and-forward consumers), `ON CONFLICT (event_id) DO NOTHING` for idempotency, and the same `ServiceMetrics` pattern for Prometheus observability.

4.2 Raw-Events Sink

Location: `src/streaming/sinks/raw_events_sink.py`

Simple INSERT into `transactions`. FK violation (missing master record) routes to DLQ topic `dlq-raw-events`. This surfaces upstream data issues rather than silently accepting orphan records.

4.3 Decisions Sink

Location: `src/streaming/sinks/decisions_sink.py`

Two write paths per event:

Path A — PostgreSQL: INSERT into `alerts` with `tier`, `fused_score`, `reasons`, `regulatory_flags`, `explanation` (JSONB). PostgreSQL failure routes to DLQ topic `dlq-decisions`.

Path B — Redis: Update risk history in client profile. INFO events are skipped. REVIEW/ALERT/BLOCK events increment `alert_{tier}_30d_count`, `alert_count_30d`, and update `last_alert_timestamp`. Redis failure is logged but does not route to DLQ (pipeline is already degraded if Redis is down).

4.4 Profile Update Rules

Tier	Behavioral State	Risk History
INFO	Updated by enrichment service	Skipped
REVIEW	Updated by enrichment service	Incremented
ALERT	Updated by enrichment service	Incremented
BLOCK	Updated by enrichment service	Incremented

Design principle: “Innocent until proven guilty.” All events update behavioral state (buffer, counters) at enrichment time because the operation actually happened (except BLOCK, which was rejected). The nightly batch job corrects any drift by excluding supervisor-confirmed anomalies during profile recomputation.

4.5 DecisionCore Output Change

`DecisionCore.build_output()` now includes `enriched_features` — the 30-feature contrast vector keyed from `models/feature_cols.json`. This was required so the decisions sink could persist the feature vector for downstream use. `DecisionCore.__init__()` loads `feature_cols.json` once at startup.

5 Infrastructure Changes

5.1 Docker Compose

Service count: 12 → 14. Both sinks added with appropriate dependencies (PostgreSQL healthy, RedPanda started). Decisions sink additionally depends on Redis. Neither sink depends on `hydrate-redis` — they don’t need pre-populated profiles.

5.2 Dockerfile

Added `ENV PYTHONUNBUFFERED=1` to `Dockerfile.api`. Without this, all Python `print()` output is buffered in Docker containers (no TTY detected), making every service appear silent in `docker compose logs`.

5.3 Prometheus

Two new scrape targets: `raw-events-sink:9103` and `decisions-sink:9104`.

5.4 Batch Scripts

Hardcoded `DB_CONFIG` removed from `batch_profile.py`, `batch_archetype_baselines.py`, and `batch_employee_profile.py`. All now import from `src.config`, enabling execution inside Docker containers via environment variables.

`hydrate_redis.py` updated to pass `REDIS_HOST` and `REDIS_PORT` from `src.config` instead of defaulting to `localhost`.

6 Debugging Journey & Findings

6.1 Stale PostgreSQL Connection

After Claude Code reset the decisions sink’s consumer group offset, the PostgreSQL connection died. The sink continued processing events but every `INSERT` failed with “connection already closed.” No error was logged because the exception was caught, and the debug print statements were not yet present.

Gap identified: The sink has no reconnection logic. A production system needs connection pooling or reconnect-on-failure.

6.2 Local vs Docker PostgreSQL Conflict

A local PostgreSQL installation running on port 5432 with password 9011361923257228 competed with Docker’s PostgreSQL on the same port (password `postgres`). Batch scripts with hardcoded credentials wrote to the local instance while `docker compose exec` commands operated on Docker’s instance. This caused “profiles exist but Redis is empty” confusion.

Resolution: Batch scripts now use `src.config.DB_CONFIG` which reads from environment variables. When run inside Docker, they connect to the correct database.

6.3 Missing Profiles After Volume Wipe

After `docker compose down -v`, the startup sequence was: `db-seed` → `hydrate-redis`. But `hydrate-redis` reads from `profile_snapshots`, which is empty because no batch job computed profiles. The streaming pipeline ran with zero profiles in Redis, causing every enrichment to produce default values (`z_amount=0`, `amount_to_mean_ratio=1`).

Gap identified: Batch profile computation is not in the Docker Compose startup sequence. Deferred to Airflow integration.

6.4 Buffer Contamination from Multiple Simulator Runs

Each simulator run appended events to `buffer:client:*` keys in Redis. Multiple runs accumulated duplicate events at overlapping timestamps, inflating 24h counters (`tx_count_24h: 25`, `cumulative_amount_24h: 306K DT`). These pathologically high feature values produced uniformly high anomaly scores.

Clean test procedure: `FLUSHDB` → re-hydrate Redis → `DELETE FROM alerts` → single simulator run.

6.5 Feature Distribution Mismatch (Open)

Even with clean profiles and clean buffers, the score distribution remains elevated (min 0.044, mean 0.899) compared to the original E2E test (min 0.002, mean 0.44). Root cause: `batch_enrichment.py` (which computed training features) uses the old 74-feature schema with different computation methods than the streaming 30-feature contrast design. The buffer-based 24h counts in streaming do not match the database-query-based 24h counts used during training.

This is the most critical open item. The Airflow integration must include a rewrite of `batch_enrichment.py` to use the same feature computation logic as the streaming enrichment. Until training features and inference features are aligned, the models will produce systematically biased scores.

7 Updated Roadmap

Priority reordered: Airflow + batch enrichment rewrite is now the most critical next step, ahead of the dashboard. The dashboard cannot show meaningful results until the feature alignment gap is resolved.

Task	Status	Priority
Sink consumers (raw-events + decisions)	Complete	—
Schema redesign (alerts, evaluation_labels)	Complete	—
Dead letter queue (basic)	Complete	—
PYTHONUNBUFFERED + batch config fix	Complete	—
<code>batch_enrichment.py</code> rewrite	Open	1 (Critical)
Airflow DAGs (nightly profile + weekly retrain)	Open	2
Streamlit dashboard	Open	3
MLflow (model tracking)	Open	4
Feedback loop (supervisor → retrain)	Open	5
Structured JSON logging	Open	6
Sink reconnection logic	Open	7

8 Files Modified / Created

File	Action
config/init.sql	Rebuilt (alerts, evaluation_labels, dropped FKs)
scripts/load_postgresql.py	Updated (split labels, removed oracle columns)
scripts/debug_single_event.py	Created (single-event pipeline comparison)
src/decision/core.py	Modified (enriched_features in output)
src/streaming/sinks/__init__.py	Created
src/streaming/sinks/raw_events_sink.py	Created
src/streaming/sinks/decisions_sink.py	Created
src/batch/batch_profile.py	Modified (import DB_CONFIG)
src/batch/batch_archetype_baselines.py	Modified (import DB_CONFIG)
src/batch/batch_employee_profile.py	Modified (import DB_CONFIG)
src/batch/hydrate_redis.py	Modified (REDIS_HOST/PORT from config)
src/streaming/simulator.py	Modified (burst_mode, offset param)
Dockerfile.api	Modified (PYTHONUNBUFFERED=1)
docker-compose.yml	Modified (14 services, sink entries)
config/prometheus.yml	Modified (sink scrape targets)